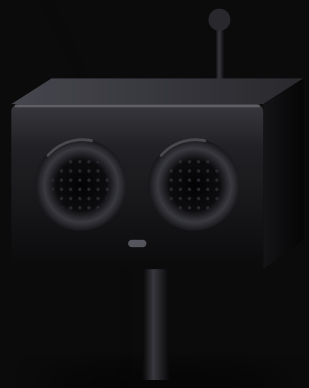
Margem



FIAP · Global Solution 2026/2 · CitizenTech · Turma 4ESOA

O aviso antes, o socorro primeiro.

Alerta comunitário de enchentes para quem mora na várzea do Rio Tietê, zona leste de São Paulo. Documento de entrega com a solução, a arquitetura e as evidências de cada área.

Integrante	RM
Italo de Souza	551500
Victor Montenegro Borges	98708
Matheus Morete Espinoso	552428
Octávio Hernandez (Chiste Cordeiro)	97894
Rafael Perussi Caczan	93092

Site (demonstração)
ryuzeen.github.io/margem-citizentech

Repositório
github.com/ryuzeen/margem-citizentech

Vídeo pitch
PENDENTE: preencher em entrega/grupo.txt

ODS
11 · 13 · 10 · 3

O problema e a solução

Quem é o cidadão

Moradores da várzea do Tietê na zona leste de São Paulo: Jardim Pantanal, Vila Itaim, Jardim Romano e Jardim Helena. Em 2009 o Jardim Pantanal ficou **90 dias debaixo d'água** (Brasil de Fato, 24/10/2020), e a área continua alagando depois de cada chuva forte. Boa parte dos moradores é idosa, usa celular simples e fica sabendo da cheia quando a água já está na porta.

O que falha hoje

- O aviso depende de alguém olhar a régua do rio: chega tarde, quando a água já passou da cota.
- Os pedidos de ajuda chegam todos juntos e em ordem de chegada. Um acamado ilhado espera atrás de um poste sem luz.
- A orientação do que fazer está espalhada em cartilhas que ninguém lê durante a enchente.

O que o Margem faz

1. **Estações IoT** com ESP32 medem nível do rio, chuva e umidade do solo e mandam leituras assinadas (HMAC) por MQTT.
2. Os dados passam por um **data lake em camadas** (bruta, confiável, refinada, quarentena), em streaming com Kafka e em lote com Spark.
3. Uma **rede neural** prevê a chance de transbordo nas próximas 2 horas. O nível final é o maior entre a régua e o modelo, no padrão da Defesa Civil.
4. Quando o risco sobe, um **agente comunicador** redige o alerta (SMS de até 160 caracteres) e ele chega ao app do morador em tempo real.
5. O morador responde "Estou seguro" ou "Preciso de ajuda" e pode escrever um relato livre. **Agentes de IA** classificam, buscam o protocolo certo (RAG) e põem idosos, PcD e acamados no topo da fila da Defesa Civil.
6. A Defesa Civil acompanha tudo num **console** com fila priorizada, check-ins, emissão de alertas e dashboard com 6 indicadores; os mesmos dados alimentam um relatório **Power BI**.

O ganho medido

dados simulados, avaliação walk-forward

222 min	82%	87%	11º
antecedência média do aviso, contra 162 min só pela régua	das enchentes avisadas com 2 h de folga, contra 24%	dos relatos na categoria certa, contra 62% por palavra-chave	posição média dos casos críticos na fila, contra 20,6º por ordem de chegada

Todos os dados (estações, histórico de 365 dias, relatos, moradores) são simulados e marcados como simulados no produto. Os números mostram o ganho do método, não desempenho em campo.

ODS atendidos

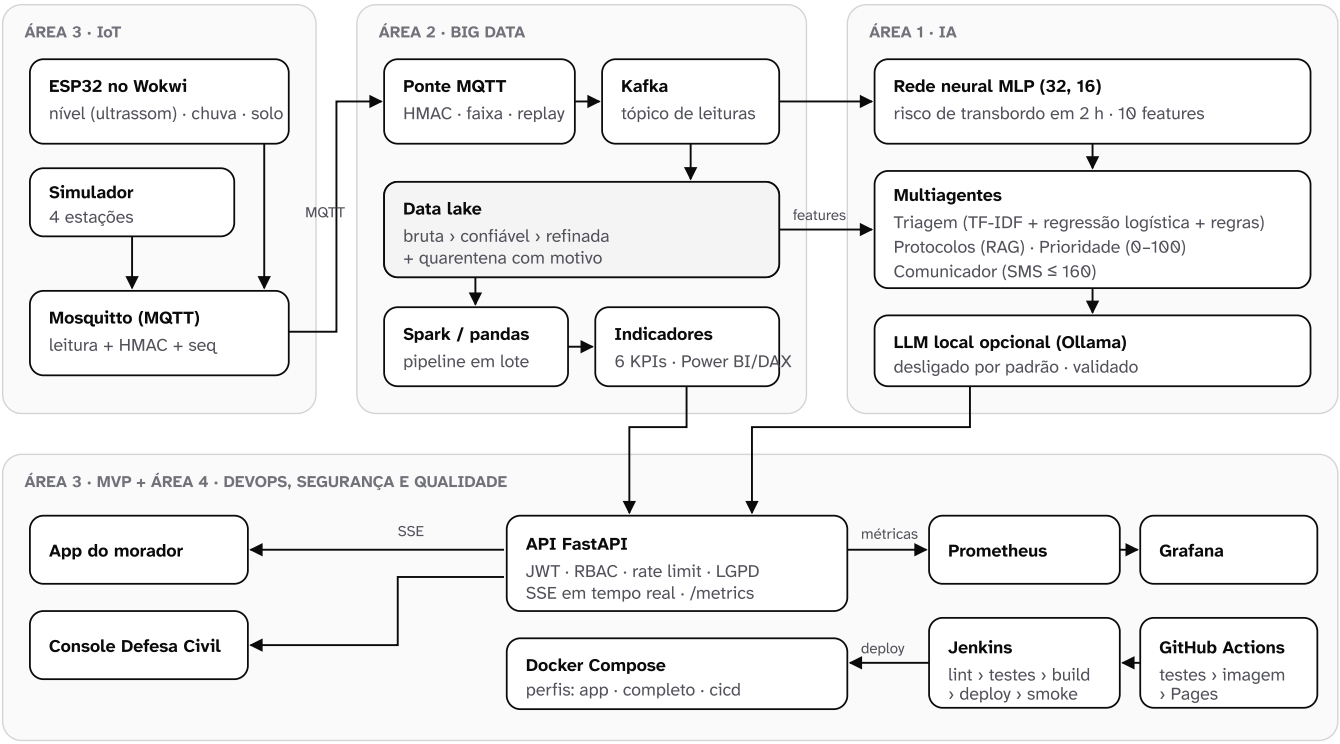
11
Cidades sustentáveis: bairros mais resilientes, aviso antes da água chegar.

13
Ação climática: chuvas extremas mais frequentes; prever é adaptar.

10
Menos desigualdade: quem mora na várzea é quem mais perde.

3
Saúde: menos gente ilhada; socorro primeiro para vulneráveis.

Uma solução, quatro áreas ligadas



Fluxo de ponta a ponta

Leitura do sensor › validação (HMAC, anti-replay, faixa física) › data lake › features › rede neural prevê o risco › se o nível sobe para *Alerta*, o agente comunicador redige o alerta › SSE para o app › morador pede ajuda ou relata › multiagentes triam › fila priorizada no console, com nome e telefone mascarados (LGPD) › indicadores no dashboard e no Power BI.

Níveis no padrão da Defesa Civil: **Observação › Atenção › Alerta › Alerta máximo**. O nível final é o maior entre a regra por cota da régua e a probabilidade do modelo (limiares 0,35 / 0,60 / 0,85).

Organização do repositório

```
area1_ia/       triagem, RAG, agentes, MLP
area2_bigdata/  histórico, MQTT/Kafka,
                pipeline, Spark, Power BI
area3_mvp_iot/  api/ (FastAPI), web/,
                iot/ (ESP32 + simulador)
area4_devops/   Jenkins, monitoramento,
                segurança, testes
comum/ config/  níveis, HMAC, território
docs/           contratos, design, erros
entrega/        PDF, .txt, pitch, evidências
```

Tecnologias × conteúdo do curso

Tecnologia	Área	Fase
Python, pandas, scikit-learn, MLP	1, 2	F1–F2
Prompts, LLM local opcional	1	F3
RAG	1	F4
Multiagentes	1	F6
Spark, Kafka, data lake	2	F4–F5
DAX / Power BI	2	F6–F7
Wokwi, ESP32, C/C++, MQTT	3	F3–F5
Segurança IoT e aplicações	3, 4	F4–F5
Docker, Jenkins, CI/CD, monitoramento	4	F4–F7

3 Área 1 de 4 · 4 pontos

IA e automação

Duas partes reais do problema: **prever o transbordo antes** da régua chegar na cota e **triar os pedidos de ajuda** para a Defesa Civil atender primeiro quem mais precisa. Tudo roda offline, sem chave de API e sem GPU.

01 · IA aplicada

1,5 pt

Previsão de transbordo (rede neural)

`StandardScaler` › `MLPClassifier(32, 16)` com early stopping, alvo `transbordo_2h`, sobre 10 features: nível, variação em 1 h e 3 h, chuva em 1/3/6/24 h, umidade do solo, % da cota de alerta e hora.

Split temporal (85/15) e validação walk-forward. Limiar pelo maior F2 (recall pesa 2×: perder uma enchente custa mais que um alarme a mais).

Triagem de relatos (ML + regras)

TF-IDF de palavras (1–2 gramas) e de caracteres (2–5, aguenta erro de digitação e gíria) › regressão logística para **categoria** (6 classes) e **gravidade** (1–5).

Léxico de vulneráveis com negação (“não tem criança aqui” não conta) e regras de segurança R1–R4 por cima do modelo, para risco de vida nunca ficar com gravidade baixa.

02 · Prompts, agentes e automação

1 pt

Agente	O que faz	Regras de automação
Triagem	categoria, gravidade e vulneráveis do relato	R1 termos de risco de vida › gravidade 4–5; R2 ilhado nunca abaixo de 4; R3 “preso em casa” › pessoa ilhada
Protocolos (RAG)	busca em 12 documentos de orientação (45 trechos, TF-IDF + cosseno)	consulta expandida com categoria e vulneráveis; máx. 2 trechos por documento; caso grave sempre começa com “ligue 193 ou 199”
Prioridade	nota 0–100 explicável no rastro	G gravidade (0–50) + V vulneráveis (0–20) + R risco do bairro (0–20) + E espera (0–10)
Comunicador	título, SMS e mensagem completa do alerta	SMS ≤ 160 caracteres; 193/199 obrigatórios em Alerta; descarta resposta do LLM que invente telefone ou endereço

Cada chamada devolve um **rastro** com o agente, o tempo em ms e um resumo da saída, mostrado ao morador e ao operador. Os prompts ficam versionados em `area1_ia/prompts/`, com entradas, formato de saída e regras de segurança. Trecho do prompt do agente comunicador (usado quando o LLM local está ligado; sem ele, o texto sai de modelos fixos com as mesmas regras):

```
Você redige alertas de enchente do Margem para moradores da zona leste de São Paulo, junto com a Defesa Civil.
Escreva em português do Brasil, em tom calmo, direto e respeitoso, com frases curtas e verbos no imperativo.
Não invente endereços, abrigos, horários nem telefones. Use apenas o abrigo informado e os telefones 193, 199, 192,
190 e 40199.
Nos níveis Alerta e Alerta máximo, cite 193 ou 199 no SMS e na mensagem completa.
Responda SOMENTE com um objeto JSON com as chaves "titulo", "mensagem_curta" e "mensagem_completa".

BAIRRO: $bairro · NÍVEL DE RISCO: $nivel · CHANCE DE TRANSBORDO EM 2 H: $prob
PONTO DE APOIO DO BAIRRO: $abrigo · AÇÕES RECOMENDADAS: $acoes
```

03 · Demonstração: entrada e saída

0,5 pt

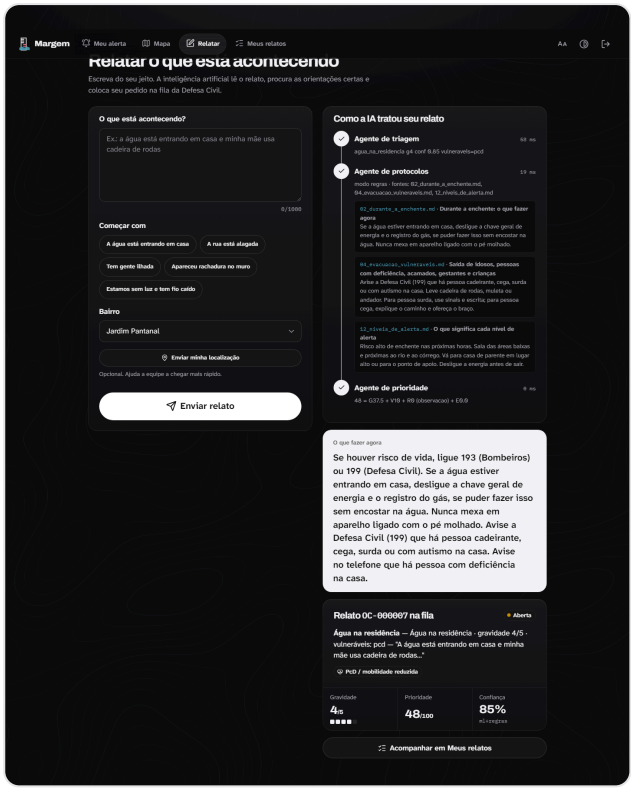
Entrada (relato do morador + contexto)

```
"minha avó acamada tá presa no quarto e a água tá
subindo,
 não consigo carregar ela sozinho"
bairro: jardim-pantanal · risco: alerta máximo (0,91)
espera: 5 min
```

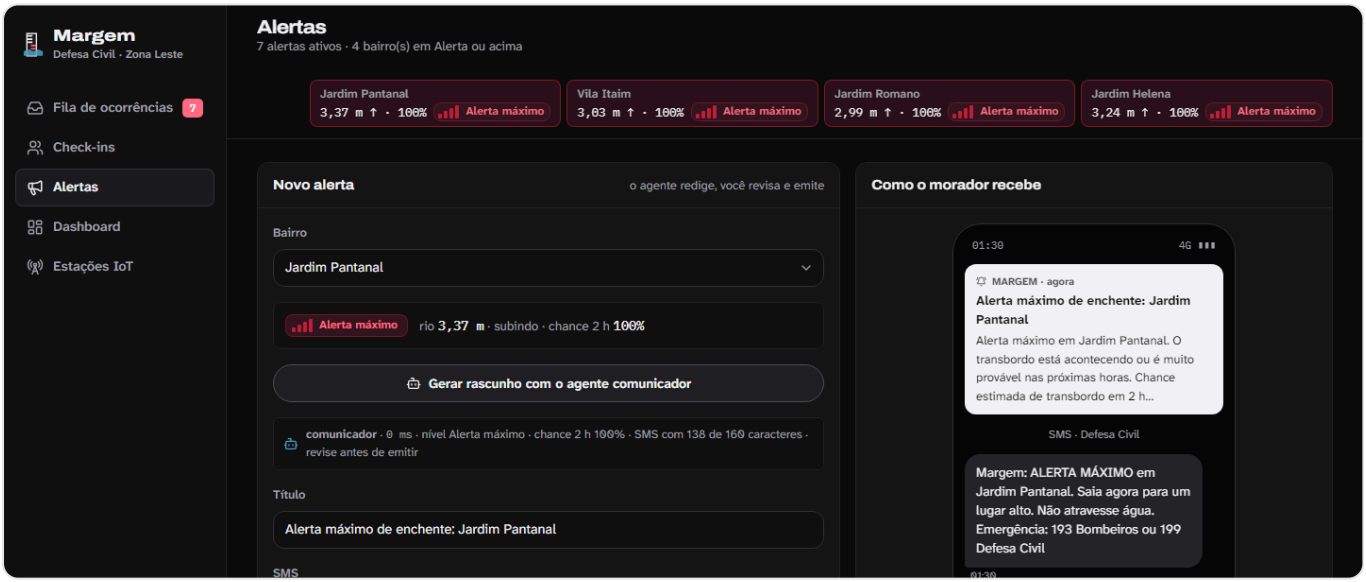
Saída dos agentes

```
categoria:  pessoa_ilhada · gravidade 4/5
vulneráveis: idoso, acamado
prioridade: 78 = G37.5 + V20 + R20 + E0.8
rastro:     triagem 2 ms › protocolos 1 ms › prioridade 0
ms
orientação: "Ligue 193. Vá para o ponto mais alto da casa
 (andar de cima, laje) levando celular, água e remédios.
 Se não der para tirar a pessoa acamada com segurança,
 ligue 193 e avise. Ajude primeiro a pessoa idosa..."
fontes:     10_pessoa_ilhada_resgate.md,
            04_evacuacao_vulneraveis.md
```

Mais 5 exemplos em `area1_ia/avaliacao/saida/exemplos_entrada_saida.json`.



No app, o relato passa pelos 3 agentes e o rastro aparece com tempo e fontes.



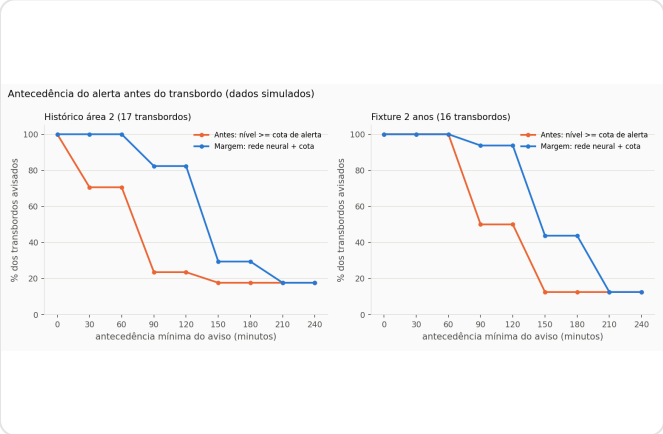
Agente comunicador: o operador gera o rascunho do alerta (SMS + mensagem completa), revisa e emite.

04 · Lógica e ganho: antes × depois

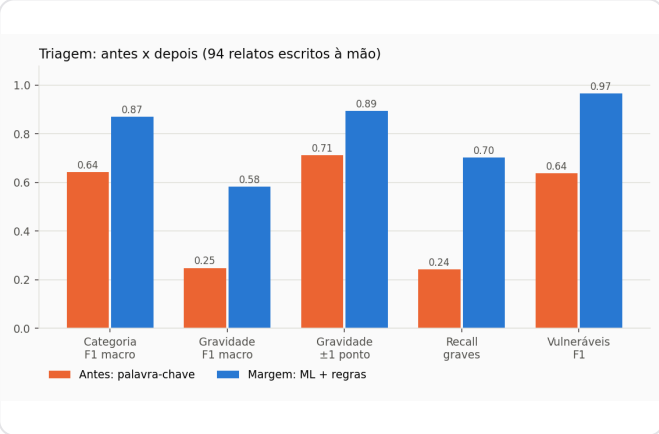
1 pt

Antes = regra por cota (avisar quando o nível passa da cota de alerta) e triagem por palavra-chave. **Depois** = Margem. Avaliação walk-forward em 243 dias (17 transbordos, 4 estações), em que cada bloco é testado com o modelo treinado só no passado; triagem testada em 94 relatos escritos à mão.

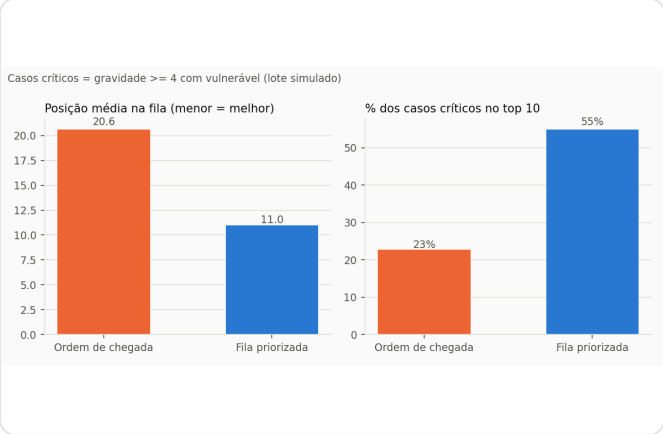
	Antes	Margem
Antecedência média do aviso	**162 min**	**222 min**
Enchentes avisadas com ≥ 2 h de folga	**24%**	**82%**
Enchentes avisadas com ≥ 1 h de folga	71%	100%
Falsos alarmes por mês (custo)	2,2	2,8
Triagem: categoria certa (acurácia)	**62%**	**87%**
Triagem: recall de casos graves (gravidade 4–5)	24%	70%
Posição média dos casos críticos na fila (30 lotes de 40)	20,6°	11,0°
RAG: documento certo no top 3 (20 perguntas)	—	100%
Tempo de triagem por relato	2–4 min (manual)	2,7 ms



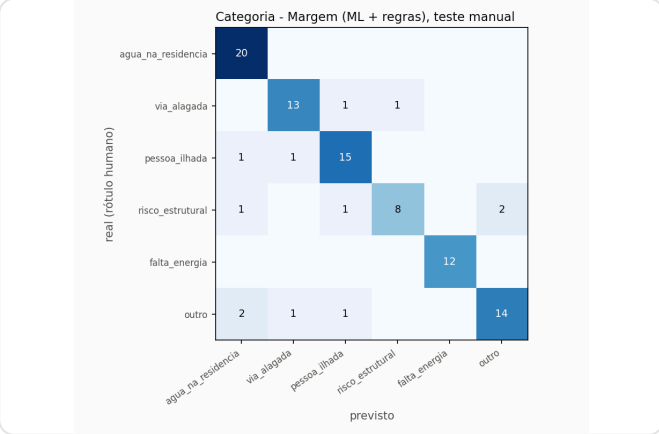
Antecedência do aviso por transbordo: regra por cota × rede neural.



Triagem: palavra-chave × ML + regras.



Fila: ordem de chegada × priorizada pela IA.



Matriz de confusão da categoria (teste escrito à mão).

Leitura honesta: em produção o Margem usa o maior nível entre a régua e o modelo, então herda os falsos alarmes da régua; o ganho é antecedência. Relatório completo em `area1_ia/avaliacao/saida/relatorio_ia.md`, regenerado com `python -m area1_ia.avaliacao`.

4 Área 2 de 4 · 4 pontos

Big Data e visualização

Os dados das estações viram informação para decidir: onde o rio vai sair, quem está em risco e se a Defesa Civil está chegando a tempo.

01 · Estrutura dos dados

1 pt

Camada	Conteúdo
Bruta	cada leitura como chegou (JSONL por dia e origem: MQTT, HTTP, histórico)
Confiável	validada (assinatura, faixa física, sem repetição), tipada e deduplicada; 1 linha por estação + horário
Refinada	leituras por hora com as features do modelo, risco por bairro, indicadores e CSVs do Power BI
Quarentena	o que foi recusado, com o motivo (`assinatura_invalida`, `seq_repetido`, `fora_da_faixa`…)



Modelo de dados (estrela) do Power BI

Tabela	Grão	Colunas principais
`fato_leitura_hora`	estação × hora	nível, variações, chuvas, umidade, % da cota, prob_2h, nivel_risco, transbordo_2h
`fato_ocorrencia`	1 por ocorrência	categoria, gravidade, prioridade, vulneráveis, status, tempos de atendimento
`fato_alerta`	1 por alerta	bairro, estação, nível, motivo, emitido_em, encerrado_em
`dim_bairro` · `dim_estacao` · `dim_morador`	1 por item	população, cotas de alerta e transbordo, vulnerabilidades (telefone mascarado)

02 · Ingestão e processamento

1 pt

1. **Coleta**: ESP32 / simulador publicam a leitura assinada no Mosquitto (MQTT), uma a cada 5 min (3 s no modo acelerado).
2. **Validação na borda**: a ponte MQTT confere HMAC, token da estação, faixa física e `seq` anti-replay. Inválido vai para a quarentena com o motivo.
3. **Streaming**: a ponte publica no **Kafka**; o consumidor grava a camada bruta.
4. **Processamento**: pipeline incremental (pandas) ou o mesmo pipeline em **Spark** gera confiável e refinada, sem duplicar ao reprocessar.
5. **Consumo**: a API lê a refinada para o modelo e o dashboard; `atualizar_dados.py` exporta os CSVs do Power BI.

```
# Job Spark no modo completo (docker compose run
spark-job)
{ "confiavel_leituras": 642,
  "leituras_hora": 68,
  "quarentena": 0 }
```

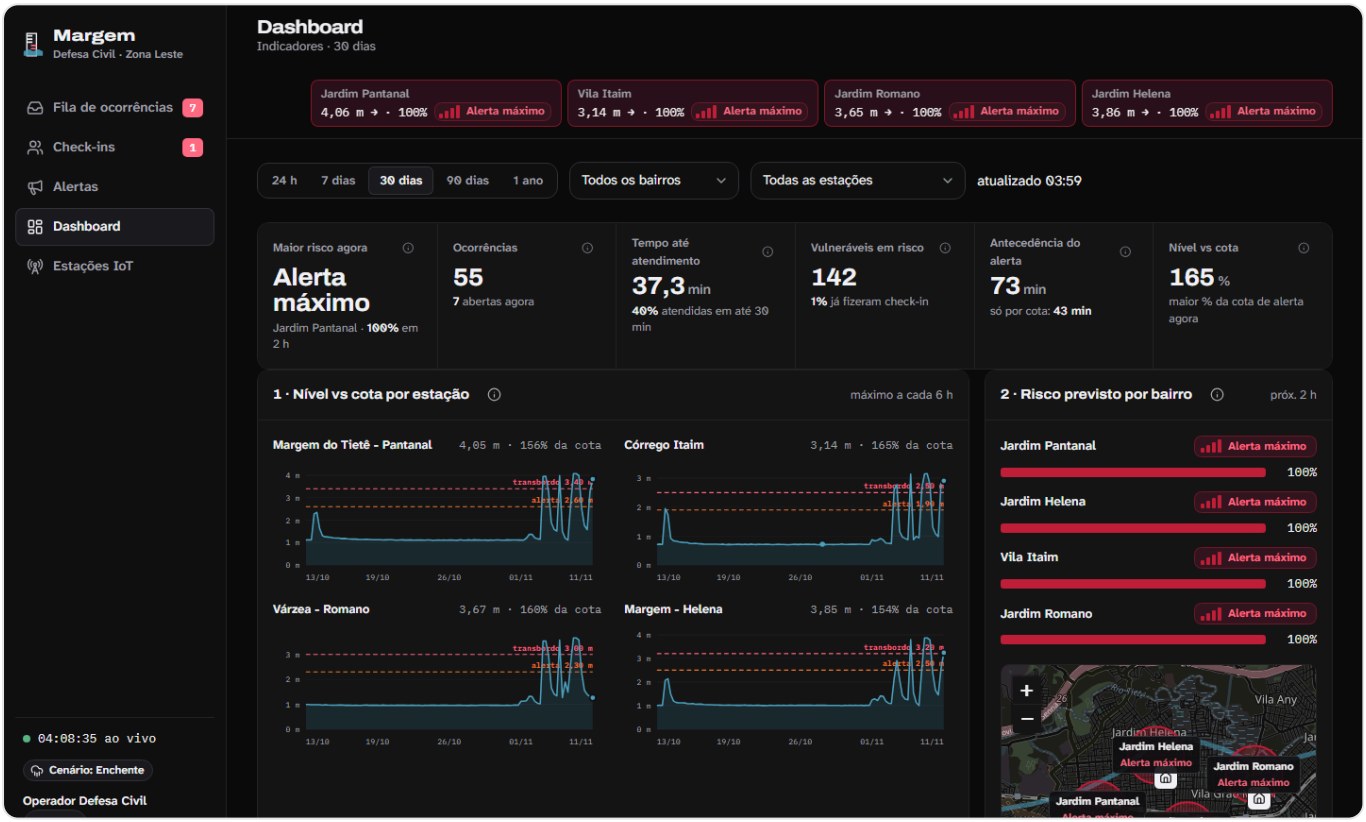
```
# Ponte MQTT recusando mensagens forjadas
EST-PANTANAL-01 -> rejeitada (seq_repetido)
EST-PANTANAL-01 -> rejeitada (assinatura_invalida)
EST-PANTANAL-01 -> rejeitada (json_invalido)
```

03 · Dashboard: 6 indicadores com filtros

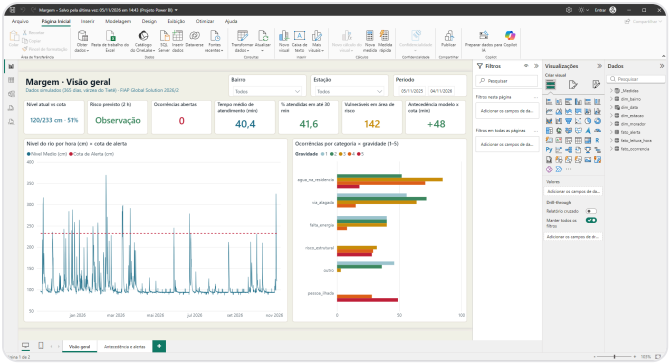
1,5 pt

No console da Defesa Civil, com filtros de período (24 h a 1 ano), bairro e estação, e um ⓘ em cada indicador mostrando a fórmula e o DAX equivalente. A decisão que cada um apoia:

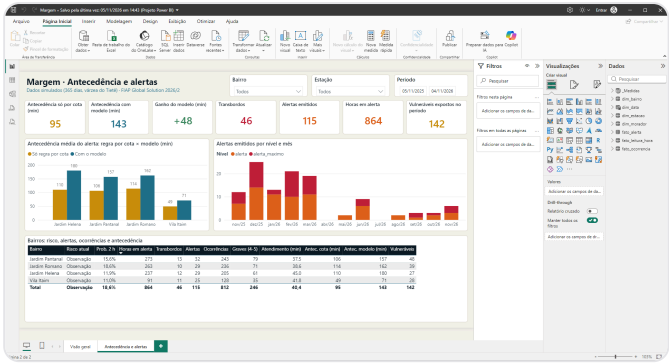
Indicador	Ajuda a decidir
1 · Nível vs cota por estação	qual estação está perto de transbordar
2 · Risco previsto por bairro (2 h)	para onde mandar alerta e equipe primeiro
3 · Ocorrências por categoria × gravidade	que tipo de equipe acionar (resgate, energia, estrutura)
4 · Tempo médio até atendimento e % em 30 min	se precisa reforçar o plantão
5 · Vulneráveis em área de risco e % com check-in	quem ainda não respondeu e precisa de visita
6 · Antecedência do alerta (modelo × régua)	se o modelo está ganhando tempo de verdade



Dashboard do console durante o cenário de enchente (filtro: 30 dias, todos os bairros). O indicador 6 mede só a janela filtrada; a avaliação completa do modelo (área 1) usa 243 dias de teste walk-forward: 222 min contra 162 min da régua.



Power BI (PBIP no repositório): visão geral.



Power BI: antecedência e alertas.

04 · Funções analíticas (DAX)

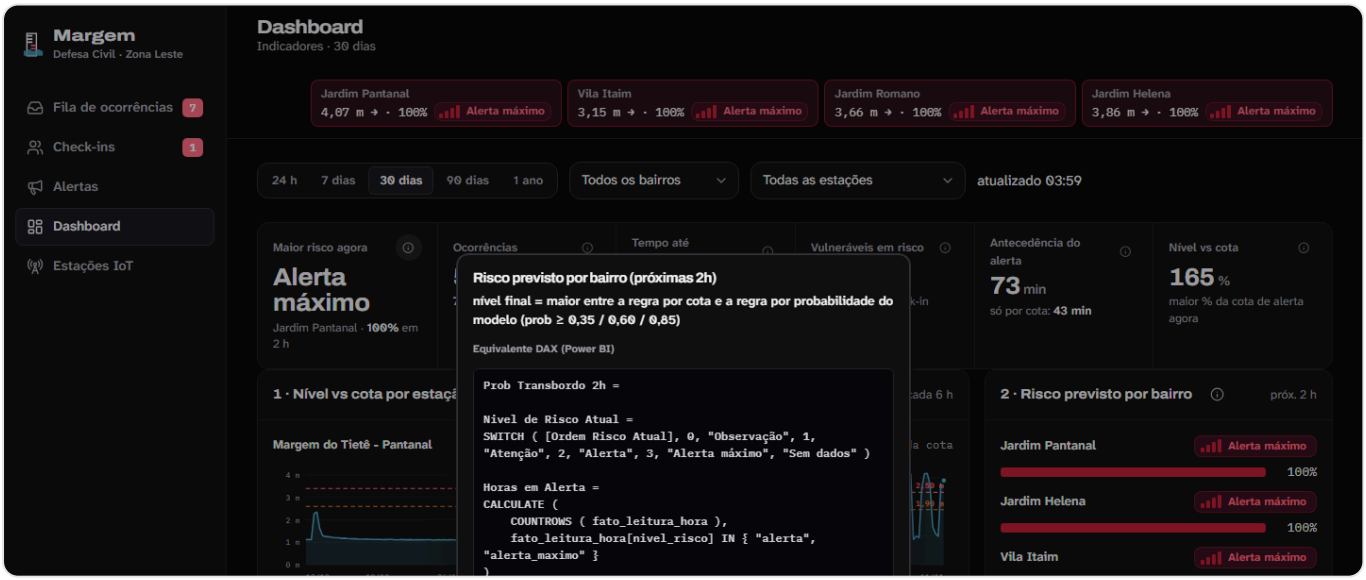
0,5 pt

26 medidas DAX no modelo semântico do Power BI, cada uma com a fórmula em Python equivalente usada no dashboard web (`area2_bigdata/analytics/MEDIDAS.md`). Exemplos:

```
Tempo Medio Atendimento (min) =
AVERAGEX (
    FILTER ( fato_ocorrencia, NOT ISBLANK ( fato_ocorrencia[atendido_em] ) ),
    ( fato_ocorrencia[atendido_em] - fato_ocorrencia[criado_em] ) * 1440 )

% Atendidas em 30 min =
VAR Atendidas = CALCULATE ( COUNTROWS ( fato_ocorrencia ), NOT ISBLANK ( fato_ocorrencia[atendido_em] ) )
VAR Rapidas   = CALCULATE ( COUNTROWS ( fato_ocorrencia ), fato_ocorrencia[tempo_atendimento_min] <= 30 )
RETURN IF ( Atendidas = 0, BLANK (), DIVIDE ( Rapidas, Atendidas ) * 100 )

Vulneraveis em Risco =
VAR BairrosEmAlerta = FILTER ( VALUES ( dim_bairro[bairro_id] ), [Ordem Risco Atual] >= 2 )
RETURN CALCULATE ( COUNTROWS ( dim_morador ),
    dim_morador[vulnerabilidades] <> "", dim_morador[em_area_risco] = TRUE (),
    KEEPFILTERS ( TREATAS ( BairrosEmAlerta, dim_morador[bairro_id] ) ) )
```



O ⓘ de cada indicador mostra a fórmula e o DAX equivalente.

5 Área 3 de 4 · 4 pontos

MVP, IoT e experiência digital

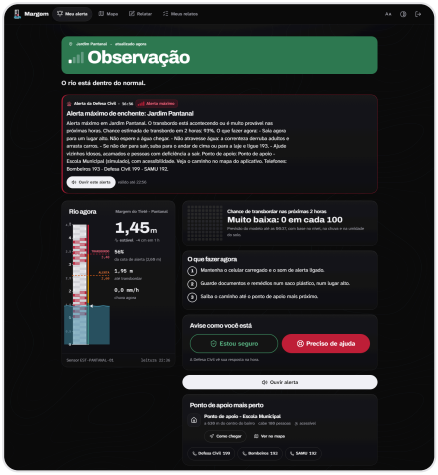
01 · Protótipo: 12 telas funcionais, web e mobile

1 pt

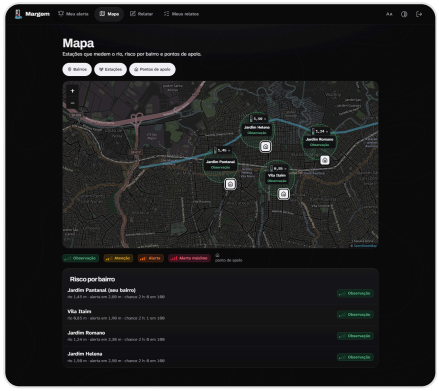
App do morador (PWA, funciona sem internet com o último estado em cache): entrar, cadastro com LGPD, meu alerta, mapa, relatar, meus relatos. Console da Defesa Civil: fila priorizada, check-ins, alertas, dashboard, estações IoT. Landing pública. Acessibilidade: letra grande, alto contraste, leitura do alerta em voz, alvos de toque de 48 px. Abaixo, uma amostra; as 25 capturas estão em `entrega/evidencias/telas_web`.



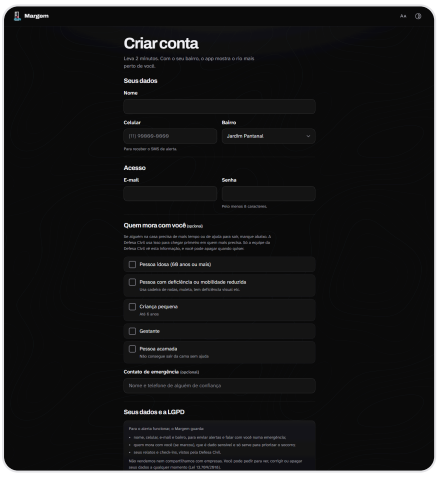
Landing pública.



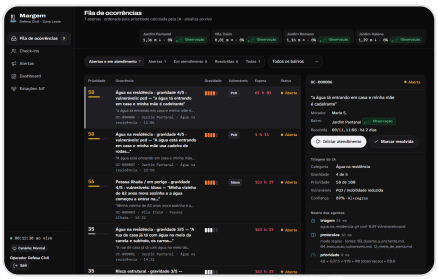
App · Meu alerta (régua, chance em 100, o que fazer, check-in).



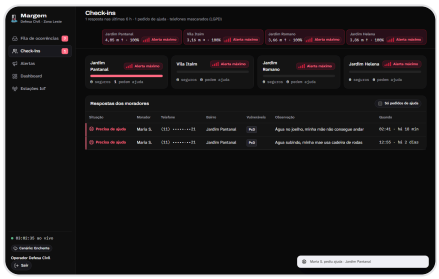
App · Mapa com estações, bairros e pontos de apoio.



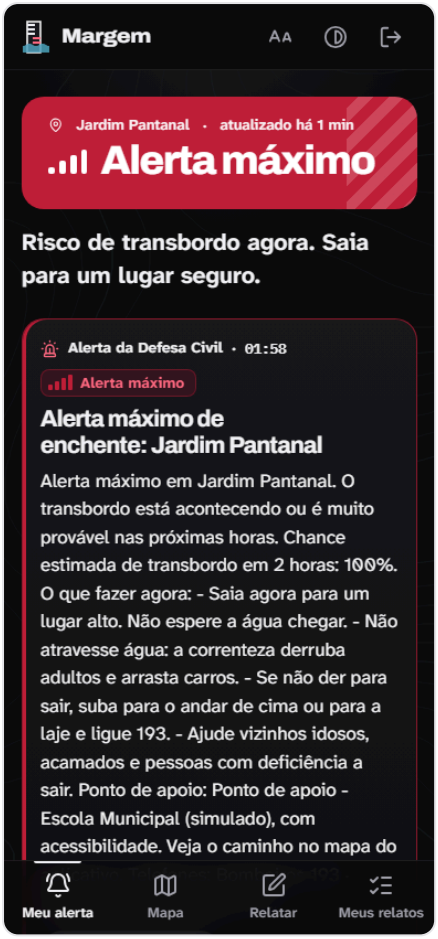
App · Cadastro com consentimento LGPD.



Console · Fila de ocorrências priorizada pela IA.



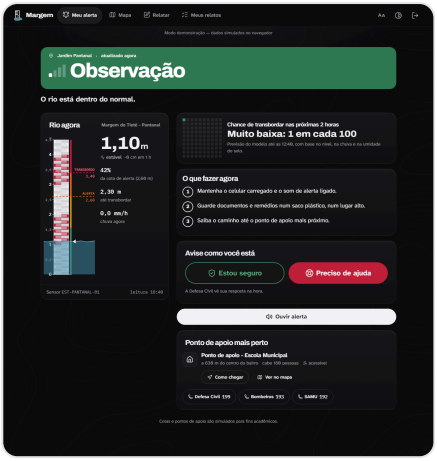
Console · Check-ins com nome e telefone mascarados.



Celular · Meu alerta.



Celular · Landing (375 px).



Modo demonstração (GitHub Pages, sem API).

02 · IoT: 3 sensores no ESP32 + simulador das 4 estações

1 pt

Componente (Wokwi)	Mede / faz	Pino
HC-SR04 (ultrassom)	distância até a água › `nível = 520 − distância` (cm)	TRIG 5, ECHO 18
Potenciômetro	pluviômetro (0–200 mm/h)	34
Slide potenciômetro	umidade do solo (%)	35
OLED SSD1306, LEDs verde/amarelo/vermelho, buzzer	mostra a leitura e toca o alarme local mesmo sem internet	SDA 21, SCL 22 · 25/26/27 · 13

O firmware (C++, `area3_mvp_iot/iot/firmware/`) publica JSON no MQTT com **assinatura HMAC-SHA256** e número de sequência anti-replay. O simulador em Python reproduz as 4 estações com os cenários normal, chuva forte e enchente, usado no modo completo e na demonstração. Projeto no Wokwi: **link pendente (preencher no README)**.

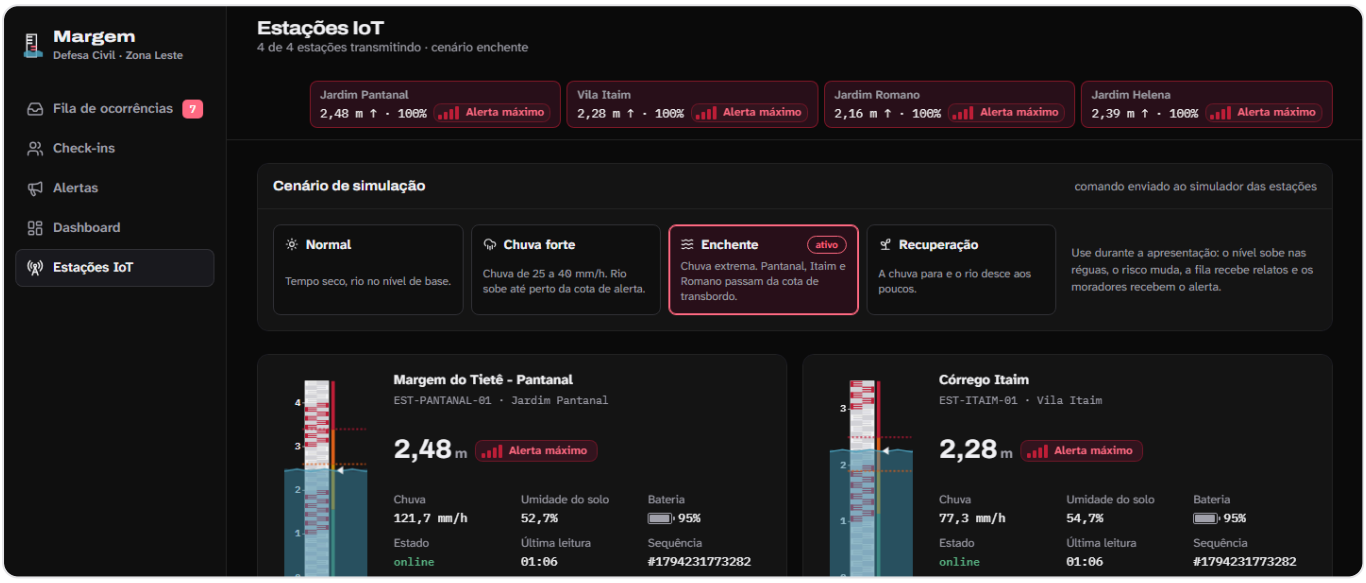

```
{"estacao_id": "EST-PANTANAL-01", "ts": "2026-11-09T02:57:05-03:00", "seq": 1794202539,
 "nivel_cm": 148.8, "chuva_mm_h": 9.4, "umidade_solo_pct": 41.7, "bateria_pct": 95,
 "firmware": "sim-1.0.0", "sig": "78a825a6a5fd7ec749dce26724d73938b769295e5f03e286e0029eda5a347d16"}
```

03 · Fluxo funcional: um uso completo, do sensor ao socorro

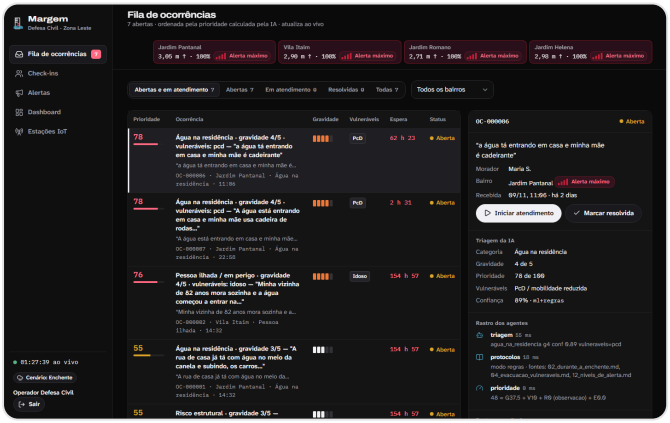
1 pt

Roteiro reproduzível pelo README (`python iniciar.py`), coberto pelo teste E2E `test_cenario_enchente_no_console_gera_alerta_no_app`:

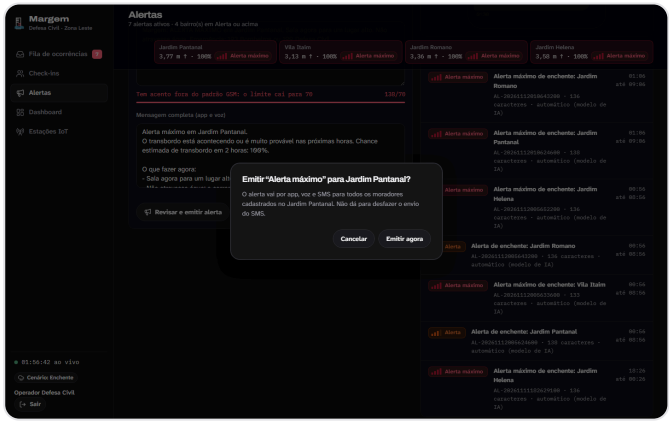
1. O operador liga o cenário **Enchente** no console. As estações mostram o rio subindo.
2. A rede neural eleva o risco e o sistema emite o **alerta automático**, que chega ao app da Maria por SSE.
3. O operador gera um rascunho com o **agente comunicador**, revisa e confirma o envio.
4. Maria recebe o alerta, com o que fazer e o ponto de apoio, e toca em **Preciso de ajuda**.
5. O pedido entra nos check-ins e o relato entra na **fila priorizada**, com dados mascarados.



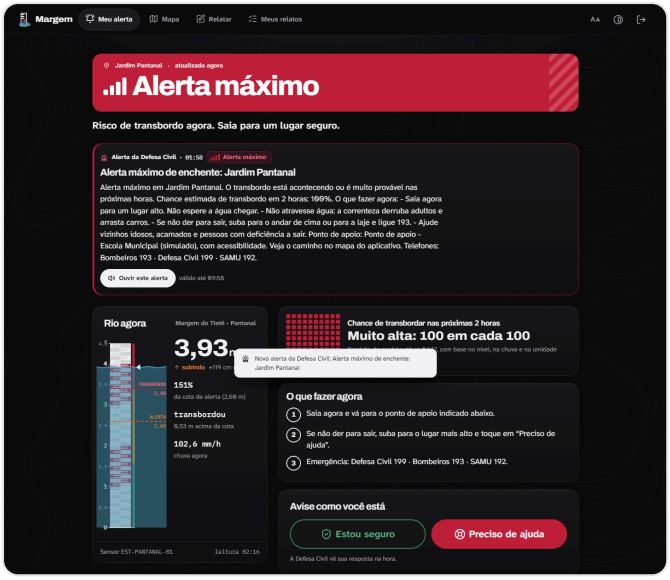
1 · Console · Estações IoT durante o cenário Enchente: nível de cada estação contra as cotas.



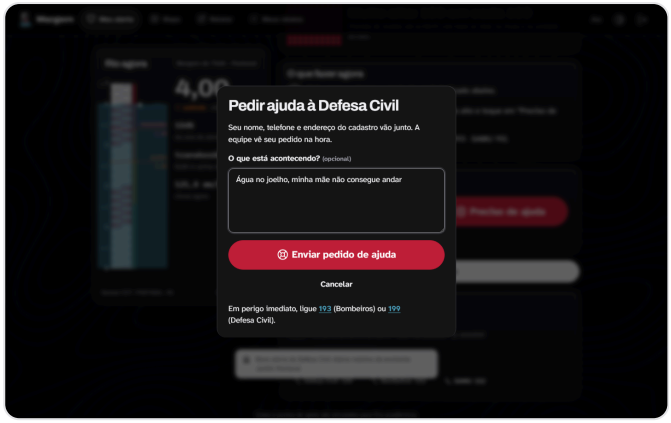
1–2 · Fila do console com os bairros em Alerta máximo.



3 · Confirmação antes de emitir (não dá para desfazer o SMS).



4 · Alerta recebido no app da moradora.



4 · Pedido de ajuda com observação.

04 · Arquitetura

1 pt

Diagrama geral com as 4 áreas e as conexões na seção 2 deste documento.

6 Área 4 de 4 · 3 pontos

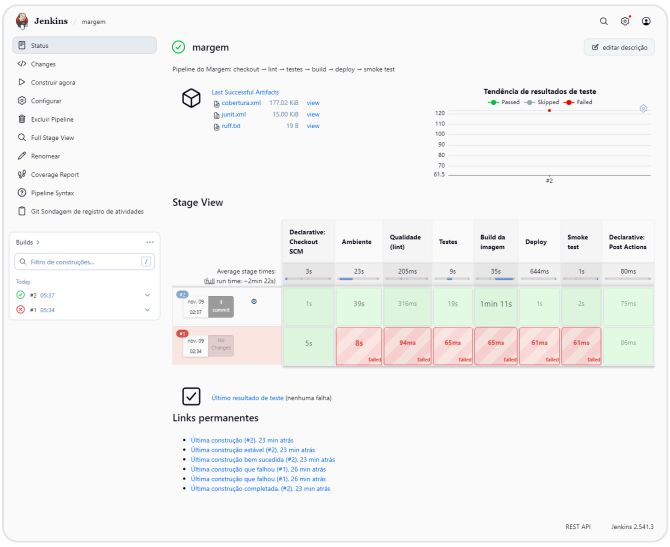
DevOps, segurança e qualidade

01 · Organização 0,5 pt

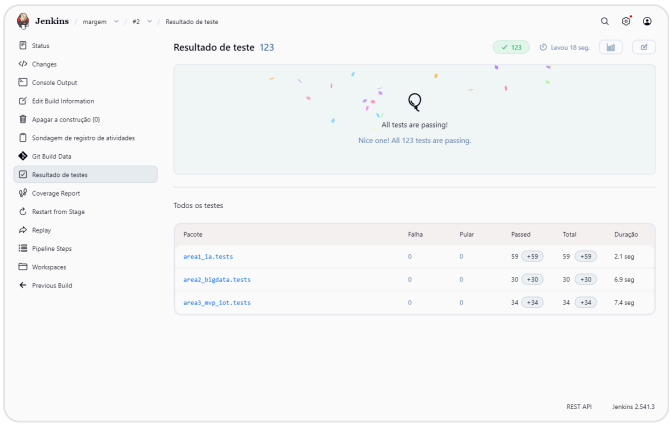
Repositório Git (`github.com/ryuzeen/margem-citizentech`) dividido por área, com README na raiz e em cada área, contratos entre áreas (`docs/CONTRATOS.md`), decisões, roadmap e registro de erros corrigidos. Configuração em `.env` (fora do Git), com `.env.example` de demonstração.

02 · DevOps: Docker, CI/CD e monitoramento funcionando 1 pt

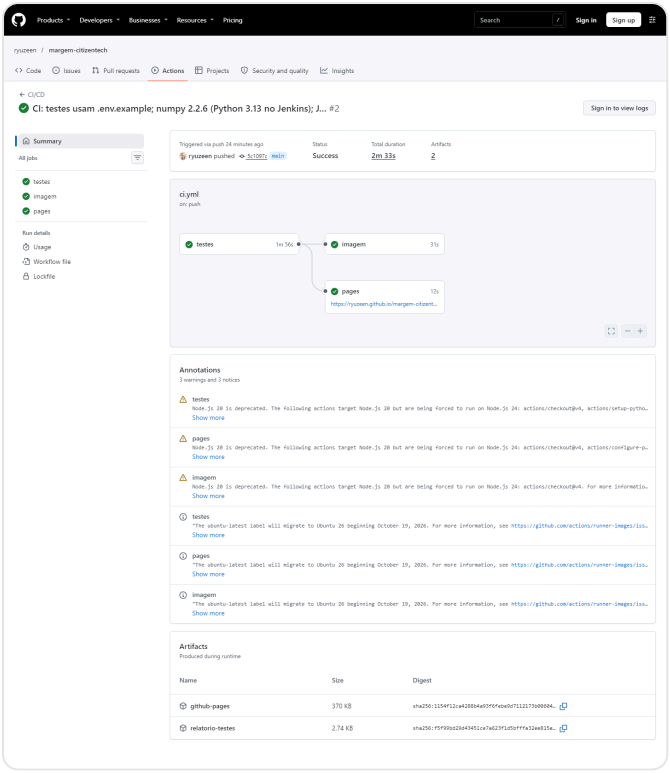
- **Docker Compose** com perfis: só a aplicação; *completo* (Mosquitto, Kafka, ponte, consumidor, simulador, Spark, Prometheus, Grafana); *cicd* (Jenkins).
- **Jenkins** configurado por código (JCasC): consulta o GitHub a cada 2 min e, a cada commit, roda ambiente › lint › testes (JUnit + cobertura) › build da imagem › deploy › smoke test, sem passo manual.
- **GitHub Actions**: testes, build da imagem e publicação do site no GitHub Pages.
- **Prometheus + Grafana**: a API expõe `/metrics` (requisições, latência, leituras aceitas e recusadas, nível por estação, risco por bairro, alertas); regras de alerta no Prometheus.



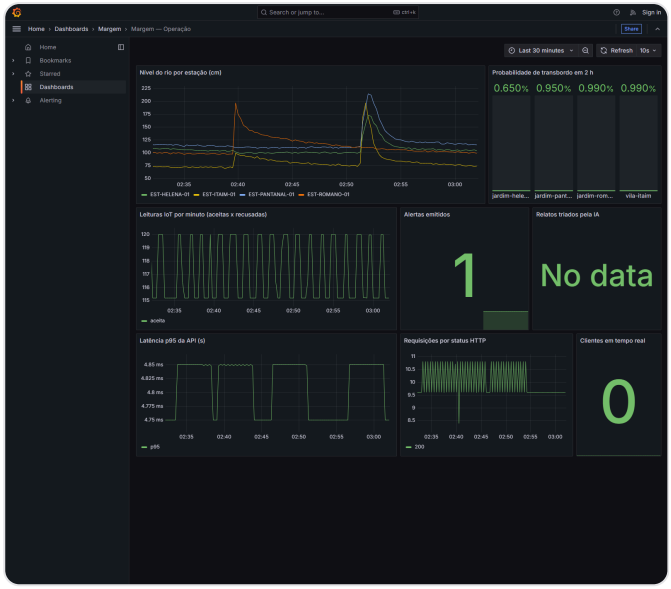
Jenkins: etapas do pipeline, build #2 verde.



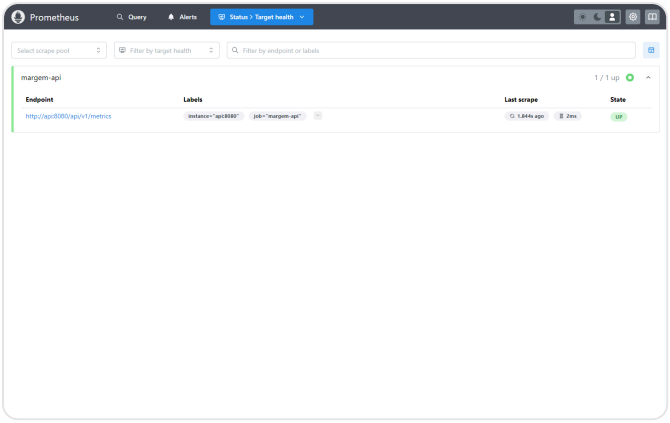
Jenkins: resultado dos testes (JUnit).



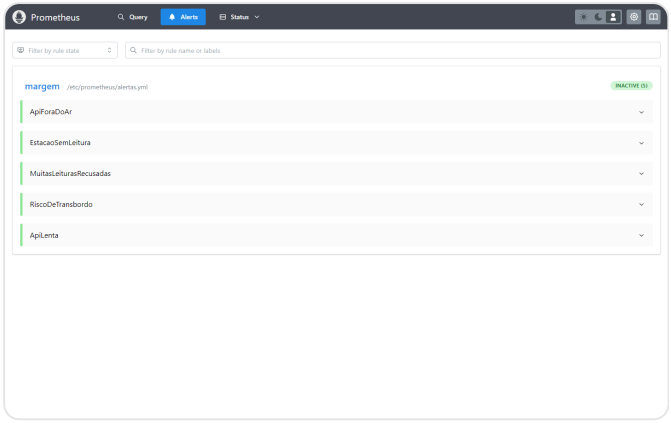
GitHub Actions: testes, imagem e Pages.



Grafana: painel “Margem” com dados ao vivo.



Prometheus: target `margem-api` UP.



Prometheus: regras de alerta.

03 · Segurança: 10 práticas

0,5 pt

Prática	Onde	Prova
Autenticação JWT (expira em 8 h, segredo aleatório no `.env`)	`api/seguranca.py`	teste automático
Senhas com bcrypt (custo 12)	`seguranca.gerar_hash`	teste automático
Autorização por papel (morador × Defesa Civil)	`exigir_papel()`	401 / 403 / 200 no roteiro manual
Validação de entrada (Pydantic, campo extra recusado)	`api/modelos.py`	422 ao mandar `"papel"` no cadastro
HMAC-SHA256 + token por estação + anti-replay nas leituras IoT	`comum/seguranca_iot.py`, firmware	replay, adulterada e token errado recusados
Limite de requisições (login, relatos, IoT)	`LimitadorTaxa`	429 depois do limite
LGPD: consentimento, dado sensível opcional, nome e telefone mascarados	`mascarar_*`	E2E: “Maria S.”, `(11) •••••-••21`
Login sem revelar se o e-mail existe (mesma resposta e tempo)	`rotas/auth.py`	teste automático
Cabeçalhos HTTP (CSP, X-Frame-Options, nosniff...)	`api/app.py`	teste automático
Segredos fora do código, contêiner sem root, SQL parametrizado	`.gitignore`, `Dockerfile`	ruff com regras de segurança (bandit)

04 · Testes: plano com 25 casos e execução

1 pt

123 testes unitários e de integração, todos passando (cobertura 75%)	**8** testes ponta a ponta no navegador (Playwright), todos passando	**25** casos no plano de testes, todos com resultado igual ao esperado	**3** defeitos achados nos testes e corrigidos

ID	Área	Caso	Resultado
CT-01	IA	Triagem melhor que palavra-chave	OK · 0,617 › 0,872
CT-02	IA	RAG devolve o protocolo certo	OK · top 3 = 1,00
CT-03	IA	Rastro dos 3 agentes e prioridade para vulneráveis	OK
CT-04	IA	Fila priorizada × ordem de chegada	OK · 20,6° › 11,0°
CT-05	IA	Rede neural antecipa mais que a régua	OK · 162 › 222 min (walk-forward)
CT-06	IA	SMS ≤ 160 e fallback quando o LLM falha	OK · 52/52
CT-07	Big Data	Inválido vai para a quarentena com motivo	OK · 14/14
CT-08	Big Data	Pipeline idempotente e incremental	OK · 8/8
CT-09	Big Data	Job Spark no modo completo	OK · 642 / 68 / 0
CT-10	Big Data / IoT	MQTT › ponte › Kafka › consumidor › bruta	OK · 2532 aceitas
CT-11	IoT / Seg.	HMAC e anti-replay barram leitura forjada (MQTT)	OK
CT-12	IoT / Seg.	HMAC, token e anti-replay na rota HTTP	OK · 202 / 422 / 422 / 401
CT-13	MVP	Alerta de ponta a ponta (automático e manual)	OK
CT-14	MVP / LGPD	Pedido de ajuda chega mascarado	OK
CT-15	MVP / LGPD	Cadastro exige consentimento (front e API)	OK
CT-16	MVP	Modo demonstração e site publicado	OK
CT-17	MVP	Mobile 375 px sem rolagem horizontal	OK
CT-18	DevOps	Pipeline Jenkins verde	OK · build #2
CT-19	DevOps	GitHub Actions verde	OK
CT-20	DevOps	Healthcheck e monitoramento	OK
CT-21	Segurança	RBAC e JWT	OK · 401/403/200/403/401
CT-22	Segurança	Rate limit e login sem vazar informação	OK
CT-23	Segurança	Cabeçalhos HTTP de segurança	OK
CT-24	Qualidade	Lint e testes automatizados	OK · 123 + 8
CT-25	Carga	10 clientes em paralelo, 1600 requisições	OK · 0 erros, p95 ≤ 60 ms

Plano completo (escopo, estratégia, ambiente, passos, evidência de cada caso e defeitos) em `area4_devops/testes/PLANO_DE_TESTES.md`; saídas em `entrega/evidencias/testes/`.

7 Diferenciais

O que vai além do pedido

Diferencial	Como o Margem atende
Áreas integradas	As leituras dos sensores (área 3) passam pelo data lake (área 2), alimentam a rede neural (área 1) e o dashboard (área 2); a API e o console (área 3) usam os agentes de IA; tudo é testado e publicado pelo pipeline (área 4).
Pipeline ponta a ponta	Um commit na `main` dispara o Jenkins (lint › testes › build › deploy › smoke) e o GitHub Actions (testes › imagem › Pages), sem passo manual.
IA medida	Antes × depois com validação walk-forward, conjunto de teste escrito à mão, tempo de triagem medido e limitações declaradas.
Validado com gente real	Entrevistas com moradores e possíveis usuários, registradas abaixo.

Validação com pessoas reais

Pendente: entrevistas do grupo

Conversar com 3 a 5 pessoas do público (moradores da várzea, agentes da Defesa Civil, familiares de idosos) e registrar em `entrega/pdf/entrevistas.md` quem são, o que disseram e o que mudou no Margem. Depois rodar `python entrega/pdf/gerar_pdf.py`.

8 Transparência

Uso de IA generativa e créditos

Ferramenta	Para quê
Claude (Anthropic), via Claude Code	apoio no planejamento da arquitetura, geração e revisão de partes do código, explicação de erros, revisão de texto da documentação e do roteiro do pitch

Todo código e texto gerado com apoio da ferramenta foi lido, executado e revisado pelo grupo antes de entrar no repositório; os resultados foram conferidos rodando os testes (123 automatizados + 8 E2E) e a aplicação. Decisões de produto, escolha do problema, validação com moradores e o vídeo pitch são do grupo. A IA **dentro** do produto (área 1) é própria e roda offline: modelos treinados por nós com scikit-learn, sem chamada a serviço externo.

Créditos

- Mapa: Leaflet (BSD-2) e dados © colaboradores do OpenStreetMap (ODbL).
- Ícones: Lucide (ISC). Fontes: Atkinson Hyperlegible Next (Braille Institute), Archivo e IBM Plex Mono (OFL).
- Contexto do problema: reportagens do Brasil de Fato sobre as enchentes do Jardim Pantanal (24/10/2020 e 14/09/2026).
- Níveis de alerta inspirados no padrão da Defesa Civil do Estado de São Paulo; orientações da base de conhecimento a partir de material público da Defesa Civil de SP, Corpo de Bombeiros PMESP, Ministério da Saúde e Cemaden.

Como rodar

```
python iniciar.py                # cria o .venv, gera dados e modelo, sobe na 8080
docker compose --profile completo up -d --build    # modo completo (MQTT, Kafka, Spark, Grafana)
docker compose --profile cicd up -d --build jenkins # Jenkins em http://localhost:8081

Morador:      maria@demo.margem.app    / Margem@2026
Defesa Civil: operador@demo.margem.app / DefesaCivil@2026
```